



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Práctica #7: Monitoreo de Uso de Memoria RAM y CPU”

Computo Integrado

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 003

Hora: N4

Salón: 3202

Equipo: 5

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
2003996	Attiana del Carmen Barrios Pérez	ITS
2018561	Mariana Cecilia Vaquera Cuellar	ITS
1962995	Carlos Abraham Gallardo Treviño	ITS
2015012	Fernanda Abigail Castillo Morales	ITS
1884405	Sergio Salazar Palacios	ITS
1960381	Andres Javier Alvarado Velez	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 30 de abril de
2024

Objetivo de la práctica

Utilizar la librería “psutil” para monitorear el porcentaje de uso del CPU y la memoria RAM de su Raspberry al usar por lo menos 3 sensores y actuadores. Cuando el uso del CPU sea mayor o igual a 65% un led amarillo debe encender, y cuando el uso de la memoria RAM sea mayor o igual a 60% debe encender un led rojo. Por último, si alguno de los dos indicadores excede el 80% de uso se deben de apagar los sensores o actuadores necesarios (puede ser 1, 2 o todos) para que bajen los indicadores. El porcentaje de uso debe mostrarse en consola o en una pantalla LCD.

Hallazgos o dificultades

Ya que utilizamos como base la práctica pasada no hubo mucha dificultad encontrada, solo conectamos todo como lo hicimos anteriormente y listo. El hallazgo que sí encontramos fue la existencia de “Stress”. Con ella, puedes generar cargas de trabajo en la CPU, la memoria, el disco y otros recursos del sistema para evaluar cómo se comporta tu sistema bajo condiciones de alta demanda lo cual fue muy útil en esta práctica porque nuestro circuito no utilizaba la memoria RAM o CPU requerida para la misma.

Código/Sketch

ARDUINO

```
#include <DHT.h>

#define DHTPIN 2      // Pin al que está conectado el sensor
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {

  float temperatura = dht.readTemperature();

  if (isnan(temperatura)) {
    Serial.println("Error");
    return;
  }

  // Se muestra solo la temperatura en el monitor serial
  Serial.println(temperatura);
```

```
    delay(2000); // Espera 2 segundos antes de realizar la próxima lectura
}
```

RASPBERRY PI

```
import RPi.GPIO as GPIO
import psutil
import time
import serial

led_amarillo_pin = 6
led_rojo_pin = 13
led_verde1_pin = 12
led_verde2_pin = 26
GPIO.setmode(GPIO.BCM)
GPIO.setup(led_amarillo_pin, GPIO.OUT)
GPIO.setup(led_rojo_pin, GPIO.OUT)
GPIO.setup(led_verde1_pin, GPIO.OUT)
GPIO.setup(led_verde2_pin, GPIO.OUT)

GPIO.output(led_verde1_pin, GPIO.HIGH)
GPIO.output(led_verde2_pin, GPIO.HIGH)

arduino_port = '/dev/ttyUSB0' # Puerto serial
baud_rate = 9600
ser = serial.Serial(arduino_port, baud_rate)

def obtener_porcentaje_uso_cpu():
    return psutil.cpu_percent(interval=1)

def obtener_porcentaje_uso_ram():
    return psutil.virtual_memory().percent

def leer_temperatura():
    try:
        ser.write(b'r')
        temperatura_str = ser.readline().strip().decode('utf-8')
        return float(temperatura_str)
    except Exception as e:
        print("Error al leer la temperatura:", e)
        return None

def controlar_leds():
```

```

porcentaje_cpu = obtener_porcentaje_uso_cpu()
porcentaje_ram = obtener_porcentaje_uso_ram()
temperatura = leer_temperatura()

if temperatura is not None:
    print("Temperatura:", temperatura, "°C")

print("Uso del CPU:", porcentaje_cpu, "%")
print("Uso de la RAM:", porcentaje_ram, "%")

if porcentaje_cpu >= 65:
    GPIO.output(led_amarillo_pin, GPIO.HIGH)
else:
    GPIO.output(led_amarillo_pin, GPIO.LOW)

if porcentaje_ram >= 60:
    GPIO.output(led_rojo_pin, GPIO.HIGH)
else:
    GPIO.output(led_rojo_pin, GPIO.LOW)

if porcentaje_cpu > 80 or porcentaje_ram > 80:
    GPIO.output(led_verde1_pin, GPIO.LOW)
    GPIO.output(led_verde2_pin, GPIO.LOW)
else:
    GPIO.output(led_verde1_pin, GPIO.HIGH)
    GPIO.output(led_verde2_pin, GPIO.HIGH)

try:
    while True:
        controlar_leds()
        time.sleep(2)
except KeyboardInterrupt:
    GPIO.cleanup()

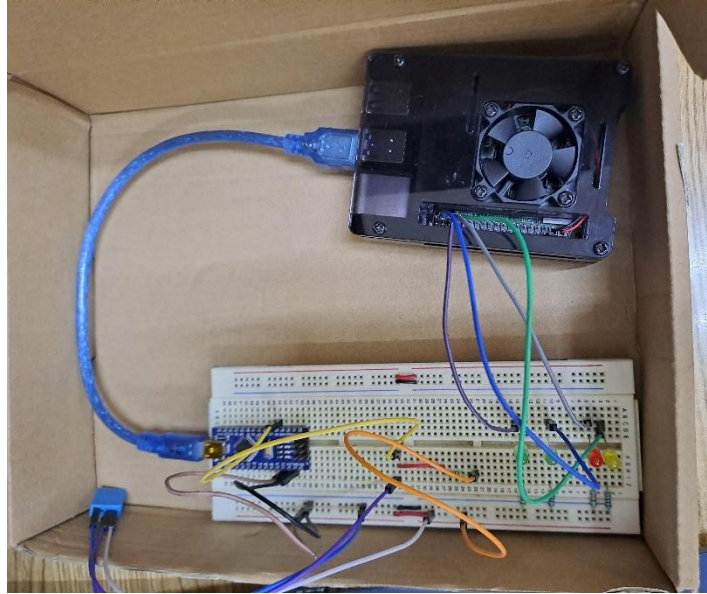
```

STRESS

stress --cpu 3 --timeout 60s

stress --cpu 4 --timeout 60s

stress --vm-bytes \$(awk '/MemTotal/{printf "%d\n", \$2 * 0.65;}' < /proc/meminfo)k --vm-keep -m 1



Conclusión general

Al incorporar la librería "psutil" y trabajar con sensores y actuadores en la Raspberry Pi, hemos logrado crear un sistema de monitoreo inteligente para el CPU y la memoria RAM. El descubrimiento de la herramienta "Stress" resultó ser un salvavidas, ya que nos permitió simular situaciones de alta demanda y evaluar cómo nuestro sistema se desempeña bajo presión. Esta experiencia nos enseñó la importancia de la planificación anticipada y la flexibilidad en el diseño de sistemas, y nos dejó con un sistema más robusto y preparado para enfrentar cualquier desafío que se presente.

Conclusiones individuales

https://drive.google.com/drive/folders/180P5NEqbc3cE0vPMsv6_IPV3OHoXcxWK?usp=sharing

Video del funcionamiento

<https://drive.google.com/drive/folders/1njoOlpN7SPemxmhKQZSzsZ3fVX6E4IIT?usp=sharing>

Bibliografía

Erik. (2015). Arduino Nano + DHT11, Temperature & Humidity sensors. Blogspot.com. Recuperado el 22 de abril de 2024 del sitio web: <https://arduino-er.blogspot.com/2015/05/arduino-nano-dht11-temperature-humidity.html>

Del Valle, L. (2017). Cómo utilizar el DHT11 para medir la temperatura y humedad con Arduino. Programarfacil Arduino Y Home Assistant. Recuperado el 22 de abril de 2024 del sitio web: <https://programarfacil.com/blog/arduino-blog/sensor-dht11-temperatura-humedad-arduino/>

Juanweb. (2023). Monitoreo del Sistema con Psutil en Python - CodigosPython. CodigosPython. <https://codigospython.com/monitoreo-del-sistema-con-psutil-en-python/#:~:text=Psutil%20es%20una%20biblioteca%20de,y%20el%20desarrollo%20de%20aplicaciones>